Access Control Testing with Burp Suite

PortSwigger Web Security Academy – Lab Report

November 11, 2026

Assessment summary

Two access control labs completed successfully. Both demonstrated that administrative actions were available through paths that were not properly protected.

1 Overview

Objective. To practise identifying and testing weaknesses that allow a normal visitor to reach administrative functions without the required permission.

Environment. PortSwigger Web Security Academy practice labs. Testing was performed only within the provided lab environments.

Tools used. Burp Suite, a web browser, and the browser's page source / website files.

2 Lab 1: Unprotected Admin Functionality

2.1 What was tested

The application exposed an administrative page even though the page was not linked through the normal user interface. The goal was to find the page and use its user-management function to remove the lab's target account, `carlos`.

2.2 Steps and evidence

1. Reviewed the site's `robots.txt` file.
2. Found the entry `Disallow: /administrator-panel`. This showed the location of an administrative page; it did not itself protect the page.
3. Opened the discovered path in the browser and reached a page listing users and delete links.
4. Used the page's delete function for the `carlos` account.
5. The application displayed a successful deletion message and PortSwigger marked the lab as solved.

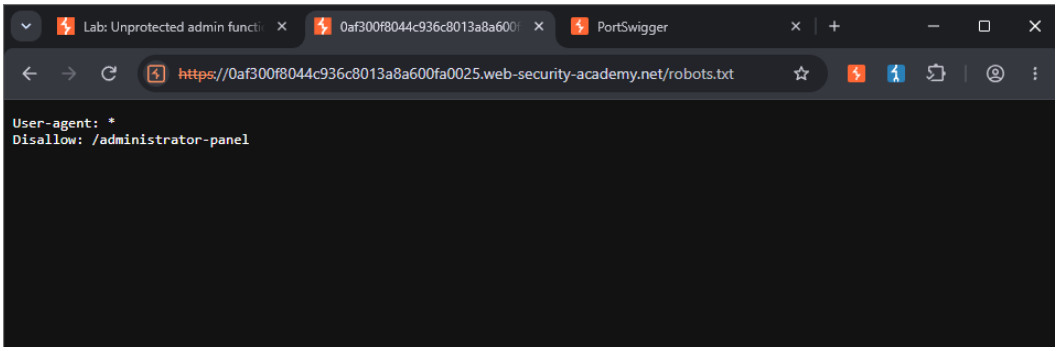


Figure 1: The `robots.txt` file disclosed the administrative path.

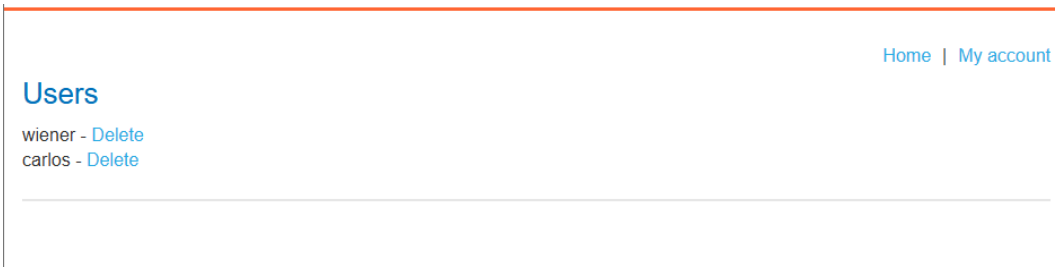


Figure 2: The administrative page displayed user accounts and delete links.

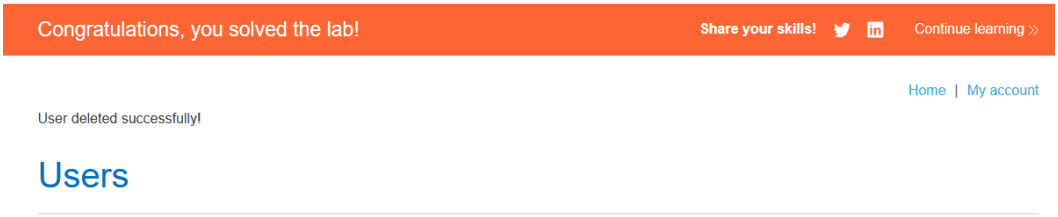


Figure 3: PortSwigger confirmation that Lab 1 was solved.

2.3 Result

The administrative page could be reached by using its disclosed path, and the target account was deleted. The lab was completed successfully.

2.4 Main lesson

A page that is not linked in the website menu is not necessarily protected. A `robots.txt` entry is a request to search engines about crawling; it is not an access-control rule. The application must check a visitor's permission on the server before showing or carrying out administrative actions.

3 Lab 2: Unprotected Admin Functionality with an Unpredictable URL

3.1 What was tested

This lab used an administrative page at a less obvious URL. The goal was to find the page through information exposed by the application, then use it to delete the `carlos` account.

3.2 Steps and evidence

1. Intercepted the home-page request and reviewed the returned page content.
2. Inspected the page's JavaScript and found a reference to the administrative path `/admin-zizvl1`.
3. Opened that path and reached the administrative user list.
4. Used the delete function for the `carlos` account.
5. The application confirmed that the user was deleted, and PortSwigger marked the lab as solved.

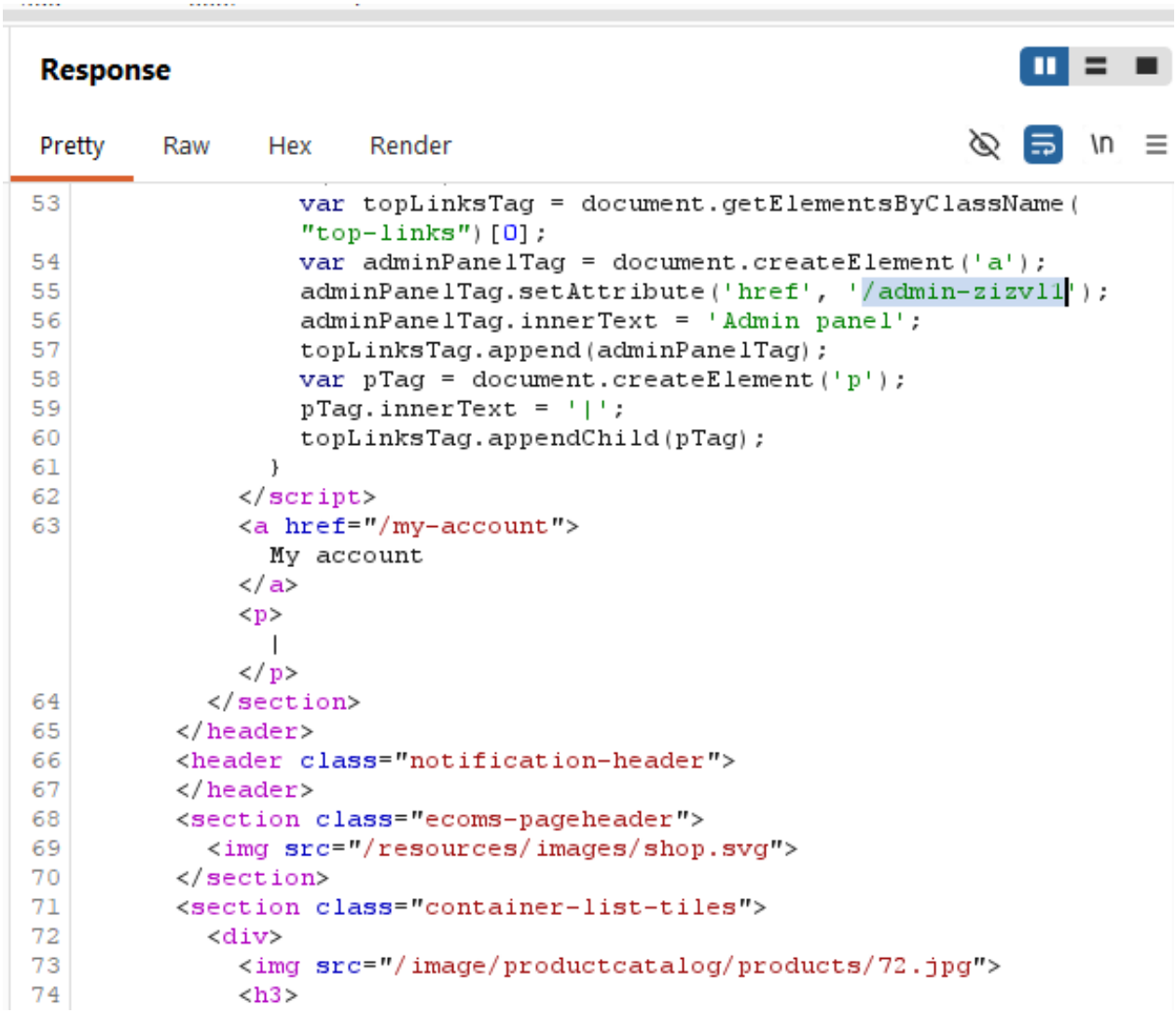


Figure 4: The returned page code contained a link to the administrative path. The request pane is omitted from this evidence image.

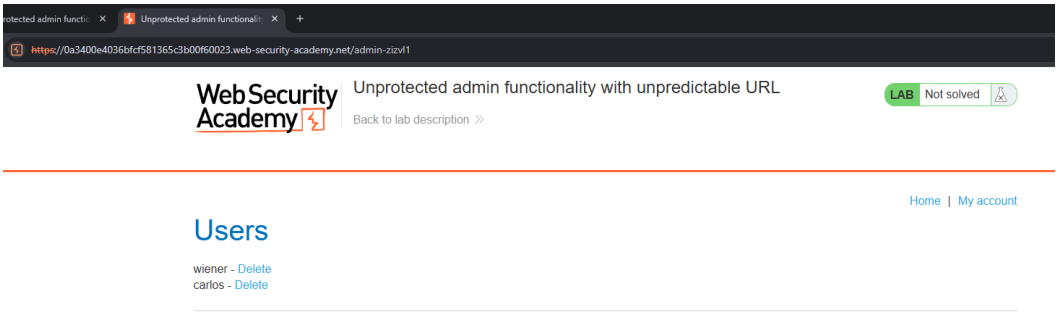


Figure 5: The administrative page was reached at the discovered, less obvious URL.

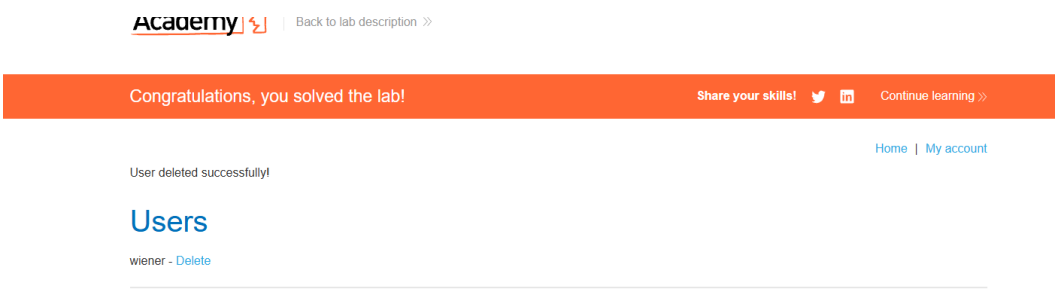


Figure 6: PortSwigger confirmation that Lab 2 was solved.

3.3 Result

The administrative path was found in the page code. The page allowed the target account to be deleted, and the lab was completed successfully.

3.4 Main lesson

An unusual or hard-to-guess URL is not a substitute for checking permissions. If a user discovers the path, the server must still prevent users without administrative rights from opening the page or performing its actions.

4 Combined Findings

Lab	How the path was found	What the test showed
Unprotected admin functionality	Disclosed in `robots.txt`	The admin page was reachable and allowed account deletion.
Unprotected admin functionality with an unpredictable URL	Found in the returned page's JavaScript	The less obvious admin page was reachable and allowed account deletion.

Both exercises show the same central problem: the application relied on the page's location being hidden or difficult to find instead of reliably checking the user's permission. In both labs, access to an administrative function was possible without the required administrative access.

5 Recommendations

- Check the user's permission on the server every time an administrative page or action is requested.
- Do not rely on hidden links, unusual URLs, or `robots.txt` to keep administrative functions private.
- Apply permission checks to actions as well as pages. For example, hiding a delete link is not enough; the server must reject an unauthorised delete request.
- Return an access-denied response when a user without the required permission requests an administrative page or action.
- Test access using both a normal user account and an administrator account to confirm that each role can perform only its permitted actions.

6 Conclusion

Both PortSwigger labs were completed successfully. The first showed how an administrative path can be revealed through `robots.txt`; the second showed how a path can be found in page JavaScript even when it is not obvious. In both cases, the administrative functions were available without a proper permission check. The key takeaway is that access must be controlled by the server, not by keeping URLs or links hidden.

Note: This report describes controlled training exercises and does not represent testing of a live third-party application.